SAMPLE

Building Tomorrow's Game Developers

The Loop's Keeper in the Machine Era

Aiman JoeHanz

Free sample: Introduction & Chapter 1

putrabooks.github.io

Buy the full book:

Ko-fi ko-fi.com/s/ae0c883ebf

Aiman JoeHanz

PutraBooks

INTRODUCTION

A game is finished not when somebody runs out of ideas but when somebody decides, on evidence, to stop changing it.

This book is about the work behind that decision: what game developers actually hold on to, how the roles differ, what the career path rewards, and what changes when the tools change. It is written for people considering the trade, for people already inside it and wondering whether they have understood it correctly, and for the people who commission games without having made one.

The claim is narrow and arguable. Game development is not the accumulation of features and it is not the execution of a vision. It is the operation of a loop: something is built, somebody plays it, something is observed, one thing is changed, and the cycle repeats until the observed behaviour matches the intended experience. Everything else in the trade, the engines, the art, the audio, the marketing, the patch notes, exists to serve that cycle or to survive its absence.

Two folklores stand in the way. The first is the auteur, for whom the game is a personal vision and the players are an audience to be brought to it. The second is the spreadsheet, for whom the numbers are the truth: retention, conversion, session length, and any experience that does not move them is a cost. Both remove the thing the loop requires, which is attention to what actually happened when somebody played. The auteur stops watching, and the spreadsheet stops seeing anything that was not instrumented.

The chapters follow the collection's order. The keeper comes first, then the territories the trade divides into, then the routes in, the working day, the craft of the loop itself, the strain it puts on people, the conversations it requires, the instruments, a composite year, and the long arc.

One warning belongs in the introduction rather than the conclusion. This trade has a documented habit of consuming the people who are most committed to it. The chapter on strain is not a digression from the craft; it is part of an accurate description, because the pressure that produces long hours comes from the same place as the pressure that produces good games, which is that the work is made of judgement applied to things people care about.

Nothing here assumes a platform, an engine, or a studio size. The mechanisms described hold for a two-person team and for a team of six hundred, and where they diverge the text says so.

The machine era arrives as a change in the instruments. Tools now generate assets, draft code, and produce variations at a rate nobody anticipated. What they do not do is play the game and notice that it is not fun yet, or notice that the third level teaches the wrong thing, or decide that the feature everyone has spent a month on should be cut. Those remain the work, and the loop is the arrangement by which they are done.

CHAPTER 1 -- THE LOOP

What the Loop Is

The loop is the smallest complete unit of game development, and it has four parts that must all be present for it to work.

Something is built, in a form crude enough to be produced quickly. Somebody plays it, ideally somebody who did not build it and who is not being polite. Something is observed, which is a narrower activity than listening to an opinion. One thing is changed, and then the whole cycle begins again with the new build.

The fourth element is the one that distinguishes the loop from ordinary iteration, and it is the deliberate limit of one change at a time. When three things change between two builds and the game feels worse, nothing has been learned. When one thing changes, the observation attaches to a cause. The trade's practitioners know this, violate it constantly under deadline pressure, and pay for the violation in confusion.

The loop is not a methodology to be adopted at a studio level. It is a discipline of the individual and small group: the designer who runs a build every afternoon rather than every fortnight, the programmer who plays their own feature before handing it over, the artist who looks at the asset in position rather than on its own. Studios that have elaborate processes and no fast loop make poor games slowly. Studios with almost no process and a fast loop occasionally make good games.

The Two Folklores

Every trade has stories that make its work legible from outside and misleading from inside, and this one has two that are unusually well known to players.

The first is the auteur. The story says that games are made by a visionary whose intent is realised by a team, that the vision is the valuable thing, and that the audience either receives it or does not. It is attractive because it is a good story, because it is occasionally true, and because it flatters the person with the idea. It fails because a vision that has not been tested against players is a guess, and games are complex enough that even experienced designers are wrong about their own work most of the time.

The second is the spreadsheet. The story says that games are products optimised against numbers: retention, conversion, session length, daily activity. It is attractive because the numbers are real, because they are legible to people who fund games, and because a studio that ignores them can run out of money. It fails because the numbers describe what happened and not why, and because a team optimising a metric will eventually produce something that moves the metric and has no other reason to exist.

Both folklores share a structure: each replaces observation with a source of authority, one inside the author and one inside the dashboard. The loop is what remains when neither authority is trusted on its own.

What Is Actually Held

Ask a practitioner what they are responsible for and the answer will vary by role, and underneath the variation is a single thing being held.

It is the correspondence between what the player experiences and what the game was intended to produce. A designer holds it for a system, an artist for a moment, a programmer for a feel, an audio designer for a mood, a producer for the whole. None of them holds the game. Each holds a claim about what somebody will feel, and the loop is the only mechanism by which that claim can be checked while there is still time to change it.

This is why the trade's central artefact is not the design document. It is the build: a thing that can be played. A design document describes an intention and cannot be wrong in an interesting way. A build can be played, and playing it produces the specific disappointment that tells you what the intention actually was.

Practitioners who have understood this stop treating documentation as the design and start treating it as the record of what has been learned. The document that matters is the one that says what was tried, what happened, and what was concluded, which is a different document from the one that says what the game will be.

The Three Commitments

This book's argument rests on three commitments, offered as a description of good practice rather than a reform.

The first is to prototype the feeling before building the system. The experience that makes a game worth playing can usually be approximated in a week with crude assets and placeholder sound. If the approximation is not interesting, the system built around it will not be either, and the discovery is much cheaper at that stage than after six months of production.

The second is to tune by playing rather than by arguing. A design question that two people disagree about is almost always answerable by a build and twenty minutes, and the answer frequently surprises both. The trade's most expensive disagreements are conducted in documents and meetings about things that could have been tried.

The third is to ship the loop that survives its players. A game is not finished when it is released. It is played for a hundred hours by somebody who finds the seams, a thousand hours by somebody who optimises the fun out of it, and in a live game it is played for years by people who will discover every mistake. Designing for the first hour and hoping is the trade's most common failure at scale.

The Speed of the Loop

The loop's value is not that it exists but that it is fast, and the speed is set by the slowest step in the cycle.

For most teams the slowest step is not building. It is deciding what to build: the feature that is specified in a document, reviewed, estimated, scheduled, and implemented a month after the idea, by which time the idea has been described so many times that it has acquired a certainty it does not deserve. A team in this condition is running a loop measured in months, which means it can make perhaps eight decisions before the project ends.

The fastest teams run the same cycle in hours. Somebody has an idea, somebody builds a crude version, four people play it after lunch, and the idea is confirmed or dead by the end of the day. That team makes two hundred decisions in the same period, and the difference is not effort. It is the arrangement.

What makes a loop fast is three things: a build that can be produced without ceremony, an audience that is available without scheduling, and a decision-maker who is present. Teams that have all three frequently describe their process as chaotic and produce coherent games, because the coherence comes from the loop rather than from the plan.

The Slow Loop and Its Symptoms

A team running a slow loop exhibits a set of symptoms that are usually diagnosed as other problems.

The first is confidence about things that have not been tested. A team that has not played the build in three weeks will describe the game's feel in the language of the design document, with adjectives nobody has verified. This is not dishonesty; it is the natural result of working from intention.

The second is disagreement that cannot be resolved. Arguments about whether a mechanic is fun, conducted by people who have not seen it, cannot be settled and so they recur. Teams in this condition describe themselves as having a design problem when they have an observation problem.

The third is the surprise at the end. A team that played the game monthly for two years knows roughly what it has. A team that played it quarterly discovers in the final month that a third of the game is not working, and the discovery is always described as sudden.

The fourth, and the most expensive, is that the cuts are made at the worst moment. Cutting in month six is a decision; cutting in month sixteen is a crisis, and the difference between the two is entirely a function of when the team found out.

Why the Loop Is Hard to Institutionalise

Every studio believes it runs a loop, and the practice is rarer than the belief for reasons that are structural rather than cultural.

The first reason is that a fast loop looks like indiscipline. Four people playing a build at eleven in the morning is not obviously work, and a manager under pressure to demonstrate progress will schedule something instead. The cost of the loop is immediate and visible; the benefit is deferred and invisible.

The second is that a playtest is a small humiliation. Somebody plays the thing you made and is bored, and the correct response is to treat that as information rather than as a verdict, which requires a composure that not everybody has. Teams where a bad session produces blame stop reporting what happened, and the loop starts producing flattering observations.

The third is that the loop requires somebody with the authority to decide. A team that plays, observes, and then takes the finding to a committee has the observation and not the decision, and the loop stalls at the point where it matters. The authority has to be in the room, and in a large studio it frequently is not.

The Loop Across a Team

The loop scales by being repeated at different levels rather than by being centralised, and the levels must be consistent with each other.

At the individual level, a designer loops on a mechanic: build, play, adjust. At the feature level, a small group loops on whether a system belongs: prototype, session, verdict. At the game level, the whole team loops on what the game is: a milestone build, a playtest with outsiders, a decision about scope.

The failure is when the levels disagree. A designer tunes a mechanic to feel good in isolation, and the feature group discovers it does not fit the game. A feature group builds a system that works, and the game-level playtest shows it makes the game worse. Each level did its job and the levels were not connected, which is why the studio's most senior designers spend their time in the middle level rather than at either extreme.

The connecting mechanism is the same build. If every level is played from the same build, the disagreements surface early and become information. If the levels are played from different builds, each is coherent internally and the incoherence is only discovered at integration, which is the most expensive place to find it.

The Observation That Is Not an Opinion

The loop's second step is where most teams substitute something easier, and the substitution is the reason their iteration does not converge.

An opinion is a reaction: it was fun, it was confusing, it felt slow. It is the cheapest thing to collect and the least useful, because it describes the person rather than the game. An observation is a description of what happened: on the third attempt the player stopped reading the prompt and walked into the wall.

Opinions are easy to solicit and impossible to act on. Observations are harder to collect, because they require somebody to watch and record, and they are the only material from which a change can be derived. A team that collects opinions discovers that its playtests produce disagreements rather than findings.

The second distinction is between what the player did and what the player said they did. People are poor reporters of their own behaviour, and a player's account of why they stopped is usually a rationalisation of the moment they got bored. The recording of the session, watched afterwards, is the primary source; the conversation is a supplement.

This is why the trade's best practitioners insist on silence during sessions and on notes taken by a person who is not playing. The arrangement is uncomfortable and it produces findings that nobody could have argued their way to.

The One Change

The fourth step of the loop is the disciplined one, and the discipline is entirely in the number.

Changing one thing per build is obvious in principle and violated constantly in practice, because a team under pressure changes everything it can think of and then finds that the

game is better without knowing why. The improvement is real and it is unusable: nobody knows which change produced it, and the next regression cannot be attributed.

The discipline has a second function that is less obvious. A single change forces the team to decide what the most important problem is, and the decision is where design judgement is exercised. A build with six changes is a build that avoided a decision; a build with one is a build that made one, and the making is the skill.

The third function is psychological. A team that changes one thing and plays it sees a small, attributable result, which is motivating. A team that changes six things and plays it sees a different game and cannot tell whether to continue, which is discouraging, and the discouragement is the beginning of the end of the practice.

When the Loop Stops Working

There is a condition in which the loop produces no more learning, and it is worth recognising because the usual response makes it worse.

The condition is that the team can no longer see the game. They have played it four hundred times, they know where everything is, and their observations are the observations of experts. Changes that a new player would find decisive become invisible, and the team begins tuning details that nobody else will notice.

The symptom is that sessions produce agreement and no findings. When six people play and nobody is surprised, the loop has stopped being an instrument and become a ritual.

The remedy is outside input, and its scarcity is why studios maintain playtest pools. A person who has never played produces forty observations in twenty minutes, most of them wrong and several of them decisive. The team's job is to distinguish, and the ability to distinguish is what four hundred sessions actually bought.

The Loop and the Machine

A tool that produces content very quickly does not produce a loop, and the distinction is the whole argument of this book applied to its newest instruments.

Generation compresses the first step. A level that took a week can be proposed in an hour, four variations in a morning. What generation does not compress is the second and third steps: somebody must play it, somebody must watch, somebody must decide. Those are bounded by human attention, which has not accelerated.

The arithmetic that follows is uncomfortable for teams that have not noticed it. If production becomes four times faster and observation stays constant, then three quarters of the new content can never be examined. A studio in that position is not making more game; it is making more unexamined game, and the trade already knows what unexamined content looks like at smaller scales.

The response is not to refuse the tools. It is to spend the gained time on the step that became the bottleneck: more sessions, more outside players, more comparison between what was proposed and what was experienced. The studios that get this right will look slower than their competitors in the first year and will ship games that are coherent, which is the only advantage that compounds.

CHAPTER 2 -- THE TERRITORIES

The Roles and What They Share

Outsiders divide game development into programming, art and design. Practitioners divide it by what has to be decided and by when, and the divisions cut across the roles.

Almost every decision in the trade is one of these: what the player is trying to do in the next thirty seconds, what they are learning, what they are feeling, and what the game does in response. A combat designer, an animator, a gameplay programmer and a sound designer are all answering versions of those four questions, which is why the trade's best teams are mixed rather than layered and why a team that hands work down a chain produces something coherent and lifeless.

The roles differ in their material. A programmer works in behaviour, an artist in form and motion, a designer in rules and pacing, an audio designer in feedback and mood, a producer in time and risk. The material dictates the loop's speed: a programmer can change a number in a minute, an animator needs a day, an art team needs a week, and an outsourcing pipeline needs a month. The fastest loop sets the pace at which the team can learn, and this is why prototypes are usually made by the people whose material is quickest.

The second thing the roles share is the build. Whatever the discipline, its work is judged in the build and not in isolation. A model that is beautiful in a viewer and wrong in the scene is wrong. A system that is elegant on paper and unreadable in play is unreadable. The trade's habit of judging work in context is the loop applied to individuals, and a studio that reviews work outside the build is reviewing the wrong thing.

The Sizes of Studio

The scale of the organisation changes the work more than the genre does, and the difference is not only one of resources.

A small team, of two to fifteen people, makes decisions in a conversation and changes the game within hours. Its constraint is capacity: everything that is not the game itself, the marketing, the contracts, the platform certification, is done by somebody who is also making the game. Its characteristic failure is to build something large and unexamined because nobody had time to play it.

A mid-size studio, of thirty to a hundred and fifty, has specialists and a process. It can sustain a project for two years and it can afford to throw work away. Its characteristic failure is the coordination cost: the loop slows because each change must pass through several people, and the game's feel is decided by the sum of decisions nobody made together.

A large studio, beyond that, has departments and dependencies. It can build things no smaller organisation could attempt. Its characteristic failure is that the loop becomes a committee, and the game is assembled from parts that were each approved and never played as a whole.

The industry's most interesting work tends to come from the middle, because it is the smallest scale at which a game can be ambitious and the largest at which a single person

can still hold the whole thing in mind. That is not a rule and it is a recognisable pattern.

The Genres and the Loop They Require

Genre is usually discussed as a marketing category and it is more usefully understood as a statement about the loop's rhythm.

An action game's loop is measured in seconds: the player acts, the game responds, the response is judged. Its craft is almost entirely in the feel of that exchange, and its tuning is done by playing the same ten seconds hundreds of times. A strategy game's loop is measured in minutes, and its craft is in the clarity of information and the legibility of consequence. A role-playing game's loop is measured in hours, and its craftsmanship is in pacing and in the accumulation of meaning.

A live-service game's loop never closes. It is measured in weeks and seasons, and its craft is retention, which is a different discipline from making a good first hour. This is why studios that move from single-player work to live operation frequently struggle: the loop they know how to run stops being the loop that determines success.

The practical use of this framing is in hiring and in scheduling. A team building an action game needs people who can tune a second, and a team building a narrative game needs people who can pace an hour. Both are designers, both are valuable, and neither can do the other's work without a period of learning that is longer than a project's schedule usually allows.

The Territories Beyond the Game

A game is surrounded by work that is not the game, and the trade's career paths run through it more often than beginners expect.

There is the platform work: certification requirements, storefront arrangements, achievement systems, accessibility conformance. It is unglamorous, it is decisive for whether the game can be released at all, and it rewards a particular temperament that enjoys completeness. There is the infrastructure work: build systems, asset pipelines, telemetry, the tooling that lets a team of sixty work on one project without destroying each other's changes. It is invisible when it works and it determines how fast the loop can run.

There is the community work, which in a live game is a design function rather than a support function, because the community's behaviour becomes part of the game's content. And there is the business work, which ranges from pitching to publishers to running a studio's finances, and which decides which games get to exist.

Practitioners who move into these territories sometimes describe it as leaving the trade. It is not. It is the same judgement applied to the conditions under which games are made, and a studio without somebody who is good at it will make fewer games than its talent deserves.

The Producer and the Schedule

Production is the discipline outsiders most often mistake for management, and its subject is not people but time and risk.

A producer answers three questions continuously: what is being built, who is available, and

what will turn out to be harder than expected. The third question is the one that distinguishes a good producer from an administrator, because it requires a technical model of the work. A producer who cannot assess whether an animation pipeline will take a month or four is a producer who transmits optimism rather than managing it.

The craft of the role is in the early warning. Games do not fail suddenly; they fail through a series of discoveries that each arrived a fortnight too late to act on. A producer's value is in creating the conditions where the discovery arrives while there is still room, which is the same argument as the loop applied to the schedule rather than the build.

The role's difficulty is that its successes look like nothing happening. A project that ships on time because a risk was identified in month four and a feature was cut in month five produces no incident, no drama, and no evidence that the producer did anything. Studios that only reward visible rescue select for producers who let things get bad enough to rescue.

The Engineer Inside the Game

Engineering in games is a distinct discipline from software engineering, and practitioners arriving from the wider industry are frequently surprised by which parts of their craft do not transfer.

The parts that transfer are rigour, structure and the ability to reason about systems at scale. The parts that do not are the standards of correctness, which in games are subordinate to the standards of experience. A physics system that is technically accurate and feels wrong is wrong. A netcode solution that is elegant and produces a millisecond of visible delay in a competitive game is a failure, regardless of its throughput.

The second difference is the budget. A frame has a fixed time, and everything the game does must fit inside it. Engineers who have worked in systems with elastic capacity find the fixed budget both constraining and clarifying: it forces an ordering of decisions that general software engineering postpones indefinitely.

The third, and the one that shapes careers, is that the work is measured by what the player notices. An optimisation that removes two milliseconds from a routine nobody perceives is worth less than a hack that makes a hit land correctly, and the trade's excellent engineers are the ones whose technical judgement includes a model of player perception.

The Artist and the Frame

Art in games is judged inside a moving image at a specific distance, and the constraints are unlike those of any other visual discipline.

The first constraint is the camera. A model is seen from the player's viewpoint, which is usually behind, above, or at eye level, and the detail that faces away from the camera is wasted work. Studios that import a film, concept art or animation pipeline without adjusting it produce beautiful assets that are half invisible, and the waste is frequently invisible to the artist and obvious to the producer.

The second is legibility. In a moving image with a busy background, the player must be able to identify the thing that matters in a fraction of a second. This is why game art's palette, silhouette and contrast rules are more aggressive than their equivalents elsewhere, and why

an asset that reads beautifully in a portfolio can be unusable in a scene.

The third is performance. Every polygon, texture and light costs a share of the frame budget, and the artist's work is a negotiation with the engineer's. Studios where that negotiation is a conversation produce art that looks expensive and runs; studios where it is a demand produce either art that does not run or technology that makes no art possible.

The Sound Nobody Notices

Audio is the discipline whose contribution is most reliably underestimated because good audio does not feel like audio. It feels like the game responding correctly.

The trade's practitioners describe the work as being about feedback and confirmation: the sound that tells a player their action registered, the ambience that tells them where they are, the mix that directs attention without the player noticing the direction. A game with careful audio is described as feeling good. A game without it is described as feeling wrong in a way that testers struggle to articulate, and the diagnosis frequently takes a week.

The economic consequence is that audio is scheduled last and cut first, which is exactly the wrong order. Its share of the budget is small relative to art and code, and its effect on the perceived quality of both is disproportionate. Studios that have learned this put an audio designer on the prototype, which is where the chapter nine slice discovered that the stealth mechanic's legibility was being carried by sound.

The craft's second property is that it is stateful. A game's audio is a system of states and transitions, and the difficulty is not in the individual sounds but in the transitions between them. This is why middleware exists and why it is a system that somebody must own rather than a library that somebody can install.

The Tester and the Boundary

Quality assurance in games is a discipline that sits between production and observation, and it is systematically undervalued by the people it protects.

A tester's work is to find where the game breaks: the sequence that softlocks a level, the save state that corrupts, the interface that cannot be navigated with a particular input. It is detail work performed under time pressure by people who are frequently employed on short contracts, and it is the reason a game can be played at all.

The discipline's second function is design information. A tester who plays the game forty times sees patterns that the team cannot: the level that is always approached the same way, the mechanic that is always ignored, the moment everybody tries the same wrong thing. Studios that employ testers as observers rather than as defect-finders get design findings from the cheapest source available.

The boundary issue is that testing is frequently a junior role, and the people doing it have the most contact with the game. Studios that treat the role as a queue for defects waste an instrument. Ones that rotate designers through it, and read what testers say about the experience rather than only about the crashes, get an outside view maintained continuously.

The Localiser and the Second Audience

ABOUT THE AUTHOR

Aiman JoeHanz is a technology writer and analyst specializing in data center infrastructure, AI networking, and next-generation computing architectures. His work spans artificial intelligence, cybersecurity, energy systems, healthcare, and infrastructure design, treating each technology as engineering: parts, failure paths, and build programs.